

Assignment on GDB, perf, valgrind

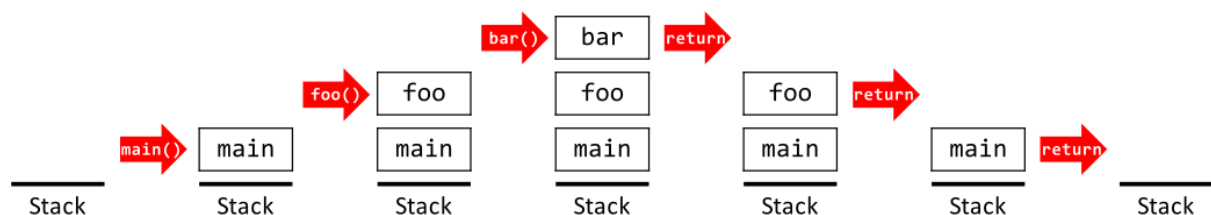
Release date: May 31st

Submission date: 9th June (Tuesday)

Corrected on: June 1, 7:10 pm.

1. **Stack visualizer:** In C and C++ (and most other languages), each function call creates an activation record, or stack frame, containing information such as local variables, saved registers, arguments, and return information. Although variables and scope are language-level concepts, their runtime implementation is closely tied to the call stack maintained by the compiled program. This task asks you to observe this mechanism directly using GDB, by inspecting the stack memory and the relevant machine registers while a program executes.

Every running program on a computer owns a piece of memory called the “stack”. The “stack” is a special area of a program’s memory that stores temporary variables created by a function. In the stack, variables are declared, stored and initialized during runtime.



Processors that follow the x86-64 instruction set have 2 special registers named “**rbp**” and “**rsp**”.

- **rbp** (base pointer) stores the address of the base (bottom) of the stack frame of the function currently being executed, while,
- **rsp** (stack pointer) stores the address of the top of the stack frame of the function currently being executed.

The stack grows towards lower addresses. This means that the addresses of values in the stack frame of the **bar** function will be lower than that in the **foo** function. It also implies that $rbp \geq rsp$.

You may think of the memory of a program as an array of bytes, then, the addresses are just the index or offsets into this array. Shown below is an example of how the stack frame may look like in the memory array. This is a made example. The real example comes below:

```
[ . . . 0x23 0x9f 0xa0 0x55 0x6b 0xff . . . ]
                ↑                ↑
            rbp                rbp
          (0x4000)          (0x4003)
```

It is possible to read the registers values and to inspect values stored at any memory location (even the stack) programmatically using the `gdb-python` API.

Read about how to trigger an activity during an event from my slides (I did not include this in the video since it was becoming long). The even that we are interested in is the `stop` event that occurs when the debugged program stops running because of a `breakpoint`, `watchpoint`, `step` or `next`. At the stop event you want to display the stack.

Your Task: You are given a skeleton pretty printer file `stack.py`. (Again, it is completely optional to use this file.) Implement a function that gets executed every time the program's execution is stopped and prints out the stack memory in the format specified below along with the points where `rbp` and `rsp` point to. Use the C file `stack_example.c` to test `stack.py`.

This is what your pretty printer should output when testing:

```
$ gcc -g -O0 -fno-omit-frame-pointer -o stack_example stack_example.c
$ gdb stack_example
```

```
(gdb) source stack.py
[stack.py] Stack visualizer loaded. It will print the stack whenever
execution stops.
```

```
(gdb) break main
```

```
...
```

```
(gdb) r
```

```
Starting program:
```

```
...
```

```
Breakpoint 1, main () at t3_program.c:19
```

```
19         return calc(5);
```

```
rsp = 0x7fffffffdbf0, rbp = 0x7fffffffdbf0
```

```
+-----+
```

```
| 01 00 00 00 00 00 00 00 | <- rsp rbp
```

```
+-----+
```

```
| 90 9d c2 f7 ff 7f 00 00 |
```

```
+-----+
```

```
| 00 00 00 00 00 00 00 00 |
```

```
+-----+
```

```
(gdb) b 11
```

```
...
```

```
(gdb) c
```

```
Continuing.
```

```
Breakpoint 2, calc (n=5) at stack_example.c:11
```

```
11         sum += i * scale + tag[0];
```

```
rsp = 0x7fffffffdba0, rbp = 0x7fffffffdb0
```

```
+-----+
| 00 00 00 00 00 00 00 00 | <- rsp
+-----+
| 00 00 00 00 05 00 00 00 |
+-----+
| 00 00 00 00 00 00 00 00 |
+-----+
| 00 00 00 00 03 00 00 00 |
+-----+
| 00 00 00 00 00 00 00 00 |
+-----+
| 88 77 66 55 44 33 22 11 |
+-----+
| 67 64 62 00 00 00 00 00 |
+-----+
| 00 a0 81 23 c4 90 1a 78 |
+-----+
| f0 db ff ff ff 7f 00 00 | <- rbp
+-----+
| f2 51 55 55 55 55 00 00 |
+-----+
| 01 00 00 00 00 00 00 00 |
+-----+
```

(gdb)

Note that the individual bytes may differ in value, but the overall structure of the output should be similar to what is given below. The stack should be printed from `rsp` to the 8-byte block starting at `rbp+16`.

2. Perf:

Background

`ConcurrentRingBuffer` is a lock-free queue designed for high-throughput logging. Multiple threads enqueue entries concurrently; a single consumer thread drains them. The implementation uses C++ atomics and raw heap allocation. It compiles and runs without errors.

Your task is not to read the code and spot bugs, it is to treat the binary as a black box and use `perf` to discover what is wrong from the hardware's perspective. Work through each question in order; each one builds on the previous.

Provided Files

You are given three files in the resources section. Do not modify any of them.

1. `ring_buffer.hpp`
2. `main.cpp`
3. `Makefile`

Build and run with:

```
make
./ipc_bench
```

perf Commands Reference

You will be using the following four commands. Run them in order they appear, each produces input you need for the next question.

Command	What it shows
<code>perf stat ./ipc_bench</code>	High level CPU counters, IPC, stalls, page faults, user/sys time
<code>perf record ./ipc_bench</code>	Samples program execution and writes <code>perf.data</code>
<code>perf report ./ipc_bench</code>	Interactive view of <code>perf.data</code> , shows overhead by function name and calling context. Maps cycle percentages to individual assembly instructions

Note: `perf record` open interactive TUI screens if not invoked with the switch `--stdio`. Press arrow keys to navigate, Enter to drill in, and q to quit.

Questions

Answer these questions in plaintext in a file called `q2.txt` inside your submission directory. No need to submit any code.

Q1 First-pass triage [4 marks]

Run `perf stat` on the binary.

```
perf stat ./ipc_bench
```

Then answer the following questions

1. The instructions line shows an IPC value (labelled **insn per cycle**). A healthy CPU-bound program typically sustains 2-4 IPC on modern hardware. What is the value you observe, and what does it tell you about where the CPU is spending its time?
2. Look at the final two lines of the output, which show user time and sys time. For a user-space lock-free data structure that makes no explicit system calls, what does a disproportionately large sys time suggest is happening inside the binary?

Q2 Locating the hot functions [5 marks]

Record the profile and open the report:

```
perf record ./ipc_bench
```

perf report

Answer the following questions:

1. Which function or library symbol is consuming the most cycles? What does its presence at the top of the list tell you about the implementation?
2. With `-O3` the compiler often inlines small functions like `enqueue()` directly into their callers, so `enqueue` may not appear as a separate symbol at all. Why does this make `perf` report alone insufficient to understand the bottleneck, and what tool do you need to go further?

Q3 Diagnosing bottlenecks [6 marks]

Record a fresh profile and open the annotated view of the producer thread:

Find the four assembly instructions that each carry a high cycle percentage and paste those lines. Then explain the following:

1. One hotspot is a memory load instruction (`mov ... (%rXX), %rYY`) that reads a variable this thread never writes. Why is reading someone else's variable expensive here? *Hint: the MESI protocol.*
2. Another hotspot is the `jne` branch that immediately follows the lock `cmpxchg`. The `cmpxchg` itself shows almost no cycles, but the branch shows >30%. Why does `perf` attribute the stall to the branch rather than to the locked instruction?
3. A third hotspot sits around a `divq` instruction. What single line of C++ source produces this instruction, and why is a 64-bit integer division a significant problem on the critical path of a ring buffer?

Q4 Understanding the atomic memory ordering flaw [6 marks]

Look at these lines inside `enqueue()` and `dequeue()` in `ring_buffer.hpp`:

```
size_t current_tail = tail_.load();
size_t current_head = head_.load();
head_.store(current_head + 1);
```

In C++, calling `.load()` or `.store()` with no explicit argument defaults `std::memory_order_seq_cst` (sequentially consistent ordering).

Answer both parts

1. Explain what `seq_cst` forces the CPU to do at the hardware level that a weaker ordering would not require. *Hint: Try looking into Store buffers and pipeline draining*
2. For a producer-consumer queue, acquire-release semantics are sufficient. Describe what acquire means on a load and what release means on a store, and explain why that pairing alone is enough to guarantee the consumer always sees fully-written data.

Note: Some concepts referenced in this question may be unfamiliar. You are encouraged to consult external resources such as documentation, online discussions, Piazza, or any LLM/tool of your choice. You may also contact Vaibhav (TA for the course) at 23b1068@iitb.ac.in if you need clarification or guidance.

Q5 Heap allocation and kernel overhead [5 marks]

Look at these two things in `ring_buffer.hpp`:

```
virtual ~LogEntry() = default;    // on the struct
LogEntry* entry = new LogEntry(); // inside enqueue()
```

Answer both parts:

1. The virtual destructor adds a hidden cost to every `LogEntry` object that is independent of the destructor body itself. What is that cost, and why does it hurt cache performance when the consumer processes entries sequentially?
2. Under heavy multi-threaded load glibc's malloc acquires internal per-arena spinlocks. Explain how this turns a supposedly lock-free enqueue into one that secretly serialises threads at the OS level, and connect this to the high sys time you observed in Q1.

Q6 Proposing and justifying the fixes [7 marks]

Based on your analysis in Q1–Q5, there are three distinct architectural flaws in `ring_buffer.hpp`. For each flaw:

1. State the flaw in one sentence.
2. State the fix in one sentence.

Hint: The 3 areas to cover are

1. *The layout of the two atomic index variables in memory.*
2. *The default memory ordering on every atomic load, store and CAS.*
3. *The dynamic allocation and virtual destructor inside the hot path*

Q7 Interpreting perf stats counters after the fixes [7 marks]

Imagine the three flaws from Q6 have been fixed and you rerun `perf stat`. For each counter below, write whether you expect it to go up, go down, or stay roughly the same compared to the flawed baseline, and explain why in one or two sentences.

Note: The reasoning behind your answer is more important than predicting the exact direction correctly. Partial credit will be awarded for well-justified explanations, even if your final prediction is inaccurate.

1. page-faults
2. instructions
3. IPC (insn per cycle)
4. sys time
5. Throughput (M ops/ sec)

Finally,

The fixed version may still show a low IPC despite a higher throughput. Why is IPC not a reliable proxy for throughput in a contention-heavy concurrent workload? What does a low IPC actually mean in this context?

3. **Valgrind:** This problem is very easy to state. You are given a program `problem.cpp`. Compile the program and run it with `Valgrind`. `Valgrind` will report problems with the program. Fix the program by adding to it not more than 5 lines of code. Do not remove a

single line from the program. Call the modified file `solved.cpp`. After you fix it, Valgrind should not complain any further.

Submission guidelines:

```
<roll-no>
|--q1
|   `--stack.py
|--q2
|   |--q2.txt
|--q3
|   `--solved.c
```

Acknowledgements: Vaibhav for Q2 and Richeek Sharma, TA for the 2019 offering of the course for Q1. I was the instructor for this offering.

3.

Acknowledgement: Thanks to Richeek Sharma for the GDB "Stack Display" question. He was a TA for the 2021 offering in which I was the instructor.